Apuntes · Clase 48 · LLM y prompt engineering

Para leer antes de los ejercicios · pensado para quien sabe Python pero nunca ha armado un prompt ni calculado una atención

Lo más importante, en 9 ideas

1. **Fluido no es correcto.** una frase bien escrita puede contradecir la fuente; se evalúan por separado.
2. **Escribir por partes.** el LLM calcula probabilidades para el siguiente token, elige uno, lo añade y repite (autorregresivo).
3. **Token y posición.** cada parte del texto es un vector con identificador y lugar; el orden cambia el sentido.
4. **Atención.** puntajes = $q \cdot k/\sqrt{d}$, pesos = softmax, salida = Σ peso × valor.
5. **Máscara causal.** al generar solo se usa lo ya escrito, nunca el futuro.
6. **Pérdida y etapas.** $-\ln(p)$; preentrenamiento, ajuste con instrucciones y uso con parámetros fijos.
7. **Ventana y temperatura.** la ventana limita los tokens; T divide los puntajes y no cambia el orden.
8. **Prompt.** instrucciones, fuente y consulta, con una salida de formato fijo y una regla para «sin evidencia».
9. **Evaluar.** controles automáticos (forma) y fidelidad (contenido); comparar prompts con las mismas consultas y reservar la prueba.

Cada sección trae una explicación en palabras simples, un **ejemplo resuelto con números distintos** a los de las diapositivas y de los ejercicios (ejemplo propio) y un resumen «para recordar». Ejercicios en `C48_Ejercicios_Cuaderno_LLM_Prompting.pdf` y notebook `C48_LLM_Prompting.ipynb`.

1 Escribir por partes: la probabilidad de una secuencia

Un LLM no decide una frase completa: decide **un token a la vez**. En cada paso la red entrega una probabilidad para cada token del vocabulario, dado todo lo escrito antes. La probabilidad de la secuencia completa es el **producto** de las probabilidades de cada paso (cada una condicionada a lo anterior). Como los productos de muchos números pequeños se vuelven diminutos, se suele trabajar con sus logaritmos: $-\ln$ del producto es la **suma** de los $-\ln$ de cada paso, que es la pérdida de la sección 7.

En palabras simples. Es como armar una frase letra por letra con un teclado predictivo: en cada paso el teclado te ofrece opciones con un porcentaje, tú (o el azar) eliges una y el teclado vuelve a ofrecer opciones con el texto ya ampliado. La probabilidad de la frase completa es qué tan «esperable» era ese camino de elecciones.

Ejemplo resuelto · tres pasos de escritura (ejemplo propio, números inventados)

Paso	Texto disponible	Token elegido	Probabilidad	$-\ln(p)$
1	«El curso»	«usa»	0.9	0.1054
2	«El curso usa»	«R»	0.6	0.5108
3	«El curso usa R»	«y»	0.5	0.6931
Total			$0.9 \times 0.6 \times 0.5 = 0.27$	$1.3093 = -\ln(0.27)$

Lectura: el modelo asigna 0.27 a esta secuencia de tres tokens. El segundo paso (0.6) y el tercero (0.5) aportan más pérdida que el primero (0.9): la secuencia es «menos segura» donde la probabilidad es menor. **Error común**: creer que 0.27 es la probabilidad de que la frase sea verdadera; es solo la probabilidad de esa secuencia de tokens bajo el modelo.

Lo que debes recordar

- Siguiente token = elegir entre todo el vocabulario con probabilidades que suman 1.
- Probabilidad de la secuencia = producto de las probabilidades de cada paso; −ln la convierte en suma.
- Alta probabilidad de una frase no significa que diga la verdad.

2 Tokens, identificadores y el orden de las palabras

El **tokenizador** parte el texto en tokens y asigna un identificador a cada uno; con ese identificador se busca el vector (embedding) del token en una tabla aprendida. Después, la red necesita saber **en qué posición** está cada token, porque un conteo de palabras no distingue «el tutor evaluó al alumno» de «el alumno evaluó al tutor».

En palabras simples. Cada token es un pasajero: su vector dice quién es; su posición, dónde se sienta. Dos autobuses con los mismos pasajeros en otro orden son autobuses distintos.

Ejemplo resuelto · tokenizar y comparar dos órdenes (ejemplo propio)

Posición	1	2	3	4	5	6	7
Token	«El»	«curso»	«usa»	«R»	«y»	«Python»	«.»
Identificador	1	2	3	4	5	6	7

Vocabulario	al	alumno	el	evaluó	tutor
Conteo de «el tutor evaluó al alumno»	1	1	1	1	1
Conteo de «el alumno evaluó al tutor»	1	1	1	1	1

Lectura: «El curso usa R y Python.» da 7 tokens (6 palabras y el punto) con un tokenizador de juguete; uno real puede partir «Python» en fragmentos y dar más. Los dos conteos de la segunda tabla son idénticos, pero las oraciones dicen cosas opuestas: solo cambian de lugar las posiciones 2 y 5 («tutor» y «alumno»). **Error común**: creer que un token es siempre una palabra; depende del tokenizador.

Lo que debes recordar

- Token: palabra, fragmento o signo; el tokenizador le asigna un identificador y de él sale su vector.
- Un conteo de palabras pierde el orden; la red añade información de posición.
- En el ejemplo de la clase H se fijó a mano: $h_1 = (1, 0, 1)$, $h_2 = (0, 1, 1)$, $h_3 = (1, 1, 1)$.

3 Ventana de contexto y presupuesto de tokens

El tokenizador parte el texto y lo convierte en una secuencia de identificadores. La **ventana de contexto** es el máximo de tokens que el modelo maneja en un uso: lo que ocupan las instrucciones, las fuentes y la respuesta cuenta junto. Planificar un prompt es hacer una resta: *ventana − instrucciones − consulta − espacio reservado para la respuesta = espacio para fuentes*. En esta guía se usa un tokenizador de juguete (cada palabra o signo de puntuación es un token); los reales parten las palabras y cuentan más.

En palabras simples. La ventana es una mesa de trabajo de tamaño fijo. Las instrucciones, las fichas y la respuesta que se va escribiendo comparten la mesa: si pones demasiado, algo tiene que salir.

Ejemplo resuelto · presupuesto de una ventana (ejemplo propio, números inventados)

Concepto	Cálculo	Tokens
Ventana del modelo	dato	200
Instrucciones	dato	60
Consulta	dato	12
Reserva para la respuesta	dato	40
Espacio para fuentes	200 − 60 − 12 − 40	88
Fichas en orden de ranking	F1 = 35, F2 = 28, F3 = 40, F4 = 22	F1 + F2 = 63; con F3 serían 103 > 88

Lectura: caben F1 y F2 (63 tokens); F3 se pasa de los 88 disponibles. Hay tres salidas: resumir F3 conservando cifras y condiciones, quitar una ficha menos pertinente o usar un modelo con ventana mayor. **Error común**: resumir F3 y perder una negación («no hay becas completas»), que invierte la respuesta.

Lo que debes recordar

- Instrucciones, fuentes y respuesta consumen la misma ventana.
- Más texto no es mejor: selecciona las fichas pertinentes y conserva cifras, fechas, negaciones y condiciones.
- Los tokens reales son más que las palabras: tus conteos son un piso.

4 De H a Q, K y V: multiplicar un estado por una matriz W

En la atención cada estado h se transforma tres veces: q = h W_Q, k = h W_K y v = h W_V. Cada transformación es una **multiplicación de un vector fila por una matriz**: cada coordenada de h multiplica una fila de W y se suman los aportes. Los pesos de W son aprendidos y quedan fijos al usar el modelo; h cambia con cada texto.

En palabras simples. Una matriz W es una receta de mezcla: dice cuánto de cada ingrediente (coordenada de h) va en cada columna del resultado. Con tres recetas distintas (W_Q, W_K, W_V) salen tres productos distintos a partir de los mismos ingredientes.

Ejemplo resuelto · h = (2, 0, 1) por una W de 3 × 2 (ejemplo propio)

Fila de W	Coordenada de h	Aporte
(1, 0)	2	2·(1, 0) = (2, 0)
(0, 3)	0	0·(0, 3) = (0, 0)
(1, −1)	1	1·(1, −1) = (1, −1)
Suma		(2 + 0 + 1, 0 + 0 − 1) = (3, −1)

Lectura: h tiene 3 coordenadas y W tiene 3 filas, así que se pueden multiplicar; el resultado tiene tantas coordenadas como columnas de W (2). Como en la diapositiva 13 ($q_3 = 1(\sqrt{2}, 0) + 1(0, 1) + 1(0, -1)$), se suman filas de W multiplicadas por las coordenadas de h. **Error común**: mezclar el orden (W h en vez de h W) o sumar coordenadas que no corresponden.

Lo que debes recordar

- q = h W_Q, k = h W_K, v = h W_V: tres matrices, tres resultados de los mismos estados.
- Mayúsculas (Q, K, V) reúnen las filas de todas las posiciones; minúsculas son un vector.
- Las W se aprenden al entrenar y quedan fijas al usar el modelo; H, Q, K, V y los pesos de atención se recalculan con cada texto.

5 Atención con números propios

La atención actualiza **una posición** mezclando información de las demás. Se hace en tres pasos: (1) comparar la consulta q de la posición con las claves k de todas las posiciones disponibles, dividiendo por √d; (2) convertir los puntajes en pesos con softmax; (3) sumar los valores v multiplicados por sus pesos. Repetimos la cuenta con otros números, tomando W igual a la identidad (así Q = K = V = H y se ve el mecanismo sin transformaciones).

En palabras simples. Cada palabra pregunta a las demás «¿quién me sirve para entenderme?». Las que más se parecen a lo que busca reciben más peso, y la palabra se reescribe como una mezcla de ellas.

Ejemplo resuelto · actualizar la posición 3 con $h_1 = (1, 0)$, $h_2 = (0, 1)$, $h_3 = (1, 1)$ (ejemplo propio)

Paso	Cuenta	Resultado
q	$q_3 = h_3$ (con W = identidad)	(1, 1)
Producto con cada k	$q \cdot k_1 = 1 \cdot 1 + 1 \cdot 0$; $q \cdot k_2 = 1 \cdot 0 + 1 \cdot 1$; $q \cdot k_3 = 1 \cdot 1 + 1 \cdot 1$	1, 1, 2
Dividir por √2	1/√2 = 0.7071; 2/√2 = 1.4142	0.7071, 0.7071, 1.4142
Exponenciales	e^0.7071 = 2.0281; e^1.4142 = 4.1133	total = 8.1695
Pesos α	2.0281/8.1695; 4.1133/8.1695	0.2483, 0.2483, 0.5035
Salida o	$\alpha_1(1, 0) + \alpha_2(0, 1) + \alpha_3(1, 1) = (\alpha_1 + \alpha_3, \alpha_2 + \alpha_3)$	(0.7517, 0.7517)

Lectura: la posición 3 mira a las tres y se mira más a sí misma (peso 0.5035) porque su clave es la que más se parece a su consulta. Como h_1 y h_2 pesan igual, la salida queda simétrica. La salida se calcula con los pesos sin redondear: con los redondeados, 0.2483 + 0.5035 = 0.7518. **Error común**: olvidar la división por √d o dividir después del softmax (cambia los pesos).

Lo que debes recordar

- Puntaje = q·k/√d; peso = softmax de los puntajes; salida = Σ peso × valor.
- Los pesos son positivos y suman 1; no son probabilidad de acierto.
- La máscara causal pone a 0 el peso de las posiciones futuras: cada posición ve solo a su izquierda.

6 Softmax y temperatura

El softmax convierte puntajes en probabilidades: exponencia cada uno y divide por el total. La **temperatura** T divide los puntajes antes del softmax: con T < 1 las diferencias se amplifican y gana más el favorito; con T > 1 se atenúan y las otras opciones tienen más oportunidad. Como T > 0 divide a todos por igual, **el orden de los tokens no cambia**.

En palabras simples. La temperatura es la perilla del azar. Abajo, el favorito gana casi siempre; arriba, todos tienen oportunidad. Pero la perilla no mejora el favorito: solo cambia cuánto se arriesga el sorteo.

Ejemplo resuelto · tres tokens A, B y C con puntajes (3, 1, 0) (ejemplo propio)

T	Puntajes / T	Probabilidades (A, B, C)	Mayor
0.5	6, 2, 0	0.9796, 0.0179, 0.0024	A: 0.9796
1	3, 1, 0	0.8438, 0.1142, 0.0420	A: 0.8438
2	1.5, 0.5, 0	0.6285, 0.2312, 0.1402	A: 0.6285

Lectura: con T = 1 la suma es $e^3 + e^1 + e^0 = 20.0855 + 2.7183 + 1 = 23.8038$, y A queda con 0.8438. Con T = 0.5 A sube a 0.9796; con T = 2 baja a 0.6285. En los tres casos A sigue siendo el primero. **Error común**: creer que una temperatura baja «hace correcta» la respuesta; solo la hace más repetible.

Lo que debes recordar

- T pequeña: más concentrado y repetible; T grande: más variado.
- T no cambia el orden, no agrega información y no garantiza la verdad.
- Elegir siempre el máximo (voraz) equivale a un sorteo con T → 0.

7 Pérdida logarítmica: cuánto «duele» no haber esperado el token observado

Si en un texto de entrenamiento aparece el token «25» después de «La beca cubre», la pérdida de la red es −ln(p), donde p es la probabilidad que asignó a «25». Si p = 1 la pérdida es 0; si p se acerca a 0 la pérdida crece sin límite. Entrenar es ajustar los parámetros para bajar esa pérdida, sumada sobre muchísimos textos.

En palabras simples. La pérdida mide la sorpresa del modelo: si esperaba el token que apareció, casi no se sorprende (pérdida cercana a 0); si apostaba por otro, se sorprende mucho y la pérdida es grande. Entrenar es aprender a sorprenderse menos con textos reales.

Ejemplo resuelto · cuatro probabilidades asignadas al token observado (ejemplo propio)

p	−ln(p)	Lectura
0.9	0.1054	casi no hay sorpresa
0.5	0.6931	equivale a 1 de 2
0.1	2.3026	el modelo esperaba otra cosa
0.01	4.6052	muy sorprendido: pérdida alta

Lectura: pasar de p = 0.5 a p = 0.1 multiplica la pérdida por más de tres (0.6931 → 2.3026). **Error común**: usar log en base 10 o 2 sin avisar: aquí es el logaritmo natural.

Lo que debes recordar

- Pérdida = −ln(p del token observado): más probabilidad, menos pérdida.
- Se aprenden los embeddings, W_Q, W_K, W_V y otros parámetros para reducirla.
- Al usar el modelo, los parámetros están fijos: lo que cambia es la entrada (el prompt).

8 Armar un prompt y revisar la respuesta

Un prompt útil para responder con fuentes tiene **tres partes**: instrucciones (qué hacer, con qué reglas y qué formato de salida), fuente (el texto de la ficha) y consulta (la pregunta). Revisar la respuesta tiene **dos niveles**: (a) controles automáticos de forma (estado, documento, referencia válida, longitud) y (b) fidelidad: ¿cada afirmación conserva los datos y condiciones de la fuente? Aplicamos el método a una ficha de la Clase 47, con otra consulta (ejemplo propio).

En palabras simples. Un buen prompt es un encargo a un practicante: tarea y reglas, documentos y pregunta, y qué hacer si no encuentra la respuesta. Revisar es abrir los documentos: primero se mira la forma del informe y después, de verdad, si lo que dice coincide.

Ejemplo resuelto · ficha D09 y la consulta «¿A qué hora son las tutorías?» (ejemplo propio)

Prompt armado.

```
INSTRUCCIONES: Responde solo con las fuentes incluidas. Conserva cifras, fechas, negaciones y condiciones. Si falta el dato, explica qué falta. Indica estado (respondible o sin_evidencia), respuesta (hasta 45 palabras) y documento (código o vacío).
FUENTE [D09] Las tutorías se realizan los miércoles de 19:00 a 20:00. Se reserva un cupo en el calendario de la plataforma.
CONSULTA ¿A qué hora son las tutorías?
```

Borrador	Estado · documento	Controles (E · D · R · L)	Fidelidad
«Los miércoles de 19:00 a 20:00.»	respondible · D09	1 · 1 · 1 · 1 (cuatro: sí)	Sí: conserva día y horario
«Los jueves a las 19:00.»	respondible · D09	1 · 1 · 1 · 1 (cuatro: sí)	No: cambia el día y omite el término
«Las tutorías cuestan 5000 pesos.»	respondible · D09	1 · 1 · 1 · 1 (cuatro: sí)	No: inventa un precio que la ficha no da

Lectura: los tres borradores pasan los cuatro controles (todos declaran respondible, citan D09 y son cortos); solo el primero es fiel. Con las cifras de este ejemplo: controles 3 de 3 = 1.000; fidelidad 1 de 3 = 0.333. **Error común**: dar por bueno un borrador porque «cita la ficha correcta».

Lo que debes recordar

- Separa instrucciones, fuente y consulta; pide un formato de salida y una regla para «sin evidencia».
- Los controles revisan la forma; la fidelidad, el contenido. Pasar los controles no certifica la fidelidad.
- Una consulta que da órdenes («ignora las reglas…») es un dato, no una fuente.

9 Evaluar un conjunto de respuestas: controles frente a fidelidad

Para tener una noción del sistema se revisan **varias respuestas** con los mismos criterios y se cuenta cuántas cumplen cada uno. Las dos preguntas son distintas: **controles** (¿tiene la forma pedida y coincide con la referencia de estado y documento?) y **fidelidad** (¿lo que afirma coincide con la fuente?). Se pueden cruzar en una tabla de 2 × 2: la casilla «pasa los controles pero no es fiel» es la que más conviene mirar.

En palabras simples. Los controles son el visto bueno del formulario; la fidelidad, el visto bueno del contenido. Un formulario perfecto con datos falsos pasa el primero y no el segundo.

Ejemplo resuelto · seis respuestas revisadas (ejemplo propio, resultados inventados)

Respuesta	R1	R2	R3	R4	R5	R6	Total
Pasa los cuatro controles	1	1	1	1	0	1	5 de 6
Es fiel a la fuente	1	1	0	0	0	1	3 de 6

	Fiel	No fiel
Pasa los controles	3	2 (R3 y R4: la casilla de riesgo)
No pasa los controles	0	1

Lectura: controles 5/6 = 0.833; fidelidad 3/6 = 0.500. Dos de las cinco respuestas que pasan los controles no son fieles. Si solo hubieras mirado los controles habrías concluido «83 % bien» y habrías pasado por alto el problema. **Error común**: usar la proporción de controles como si midiera la calidad del contenido.

Lo que debes recordar

- Controles = forma y coincidencia con la referencia; fidelidad = contenido frente a la fuente.
- Pasar los controles no certifica la fidelidad: C02, C04 y C06 de la clase lo muestran.
- Un conjunto de respuestas construidas describe el ejercicio; no estima el rendimiento de un LLM.

10 Comparar prompts sin engañarse

Para saber si agregar ejemplos mejora el prompt, se comparan dos variantes con **las mismas consultas, el mismo modelo y la misma configuración**, y se aplican los mismos criterios a las respuestas. Una respuesta aislada no demuestra una mejora general. Además, la comparación y el ajuste se hacen con un conjunto de **desarrollo y validación**; las consultas de la **prueba final** se reservan.

Ejemplo resuelto · dos variantes en 8 consultas de validación (ejemplo propio, resultados inventados)

Variante	Respuestas fieles por consulta (1 sí, 0 no)	Fieles	Proporción
A: con fuentes	1 1 0 1 1 0 1 1	6 de 8	0.750
B: con fuentes y ejemplos	1 1 1 1 1 0 1 1	7 de 8	0.875

Lectura: B es mejor en una consulta de ocho (la tercera): una diferencia de 1/8 = 0.125. Con tan pocas consultas no basta para afirmar que los ejemplos mejoran el prompt en general; hay que repetir con más consultas. Y si hubieras corregido el prompt mirando la consulta 3, esa consulta ya no sirve como prueba final. **Error común**: cambiar tres cosas a la vez y atribuir el efecto a una.

Lo que debes recordar

- Cambia una cosa a la vez; mismas consultas, mismo modelo, misma configuración, mismos criterios.
- Reserva consultas nuevas para la prueba final; si guiaron el prompt, pasan a ser desarrollo.
- RAG = buscar (embeddings) + redactar (LLM) + comprobar (controles y fidelidad); se evalúan las dos partes.

11 Lista de revisión y glosario

Antes de confiar en una respuesta de un LLM

- ☐ ¿El prompt separa instrucciones, fuente y consulta, y dice qué hacer si falta evidencia?
- ☐ ¿La fuente incluida contiene el dato que se pregunta (o la respuesta correcta es «sin evidencia»)?
- ☐ ¿Se conservan cifras, fechas, negaciones y condiciones de la fuente?
- ☐ ¿El estado, el documento citado y la longitud coinciden con lo pedido?
- ☐ ¿Cada afirmación está respaldada por el texto de la ficha citada (fidelidad)?
- ☐ ¿La consulta intentó dar órdenes? Se trata como dato, no como fuente.
- ☐ ¿Se evaluó con consultas distintas de las que guiaron el prompt?

Término	Significado en esta clase
LLM	Large Language Model: modelo de lenguaje de gran escala (muchos parámetros).
Prompt / prompt engineering	La entrada con la que se plantea la tarea / preparar y revisar esa entrada para una tarea concreta.
Token y tokenizador	Parte del texto (palabra, fragmento o signo) / quien divide el texto y asigna un identificador a cada parte.
Embedding	El vector inicial de un token, buscado en una tabla aprendida durante el entrenamiento.
Estado h y matriz H	Vector de la posición j que recibe la atención / matriz que reúne los estados como filas.
Atención y autoatención	Operación que combina información de varias posiciones con pesos / cuando las posiciones son de la misma secuencia.
Q, K, V y W	Consultas, claves y valores / matrices de pesos aprendidos que los producen a partir de H.
Máscara causal	Bloquea el uso de posiciones futuras al predecir el siguiente token.
Transformer	Red que combina atención y transformación de cada posición en bloques que se repiten.
Autorregresivo	Generar usando lo ya escrito para continuar.
Ventana de contexto	Límite de tokens que el modelo maneja en un uso: instrucciones, fuentes y respuesta.
Temperatura (T)	Divide los puntajes antes del softmax: cambia las probabilidades, no el orden.

Término	Significado en esta clase
Pérdida	−ln(p) del token observado; entrenar es reducirla.
Preentrenamiento / ajuste con instrucciones	Aprender a continuar textos / aprender con pares de instrucción y respuesta deseada.
respondible / sin_evidencia	Estados de la salida: la fuente permite contestar (aunque sea «no») / ninguna ficha entrega el dato.
Controles y fidelidad	Revisiones de forma (estado, documento, referencia válida, longitud) / revisión del contenido frente a la fuente.
RAG	Retrieval-Augmented Generation: buscar fichas, redactar con su texto y comprobar.